数据库概念

第一章

数据

描述事务的基本符号，是数据库的基本对象。

数据库

定义：互相关联的数据集合 或 长期储存在计算机内、有组织的、可**共享**的大量数据集合。

数据库管理系统(DBMS)

定义：由一个**互相关联的数据的集合**（数据库）和一组用**以访问这些数据的程序**所组成。

核心功能：**数据定义**、**数据操作**、**数据安全**、**数据恢复和备份**、**并发控制**、**事务管理**。

数据库系统应用

定义：由数据库、数据库管理系统、应用系统、数据库管理员（或用户）所组成。

数据抽象

- 物理层

  最低层次的抽象，描述数据是怎样存储的

- 逻辑层

  描述数据库中存储了什么数据以及这些数据间存在什么关系。逻辑层用户不必知道物理层结构，保证了物理数据独立性

- 视图层

  最高层次抽象，只描述整个数据库的某个部分。

物理数据独立性

指用户的应用程序与数据库中数据的物理存储是相互独立的

逻辑数据独立性

指用户的应用程序与数据库的逻辑结构是相互独立的

> 数据独立性：不想修改应用程序

数据库系统模式

实例：特定时刻存储的数据库中的信息集合。

数据库模式：数据库总体的设计。有**物理模式**、**逻辑模式**、**子模式**。

> 数据库系统模式就是变量类型，实例就是该变量在某个特定时间的值

数据定义语言DDL

数据存储和定义，定义了数据库模式的实现细节

数据操纵语言DML

使用户可以访问或操纵数据库数据

过程化、非过程化

过程化：指定获取什么数据、如何获取数据

非过程化：只指定获取什么数据

存储管理器

负责管理和存储数据库中的数据。

负责将原始数据通过操作系统提供的文件系统存储在磁盘上。

数据文件：存储数据库本身

数据字典：存储关于数据库结构的元数据，特别是数据库模式

事务管理

事务：数据库应用中完成单一逻辑功能的操作集合

ACID：即**原子性**、**一致性**、**隔离性**和**持久性**

- 原子性：事务中每个操作要么完全执行，要么完全不执行
- 一致性：确保数据库从一个有效状态到第二个有效状态
- 隔离性：并发执行时，不受到其他事务的影响
- 持久性：操作的更改是永久性的

恢复管理器：保证原子性、持久性

并发控制管理器：保证数据库的一致性

事务管理器：包括恢复管理器、并发控制管理器

第二章

关系数据库的结构

关系 用来指代表，**元组** 用来指代行，**属性** 用来指代表中的列

关系实例表示一个关系的特定的实例，即所包含的一组**特定的行**

域：关系中的属性允许取值的集合

关系模式的所有属性的域 是***原子的，即不可再分的***

> 关系 <-> 变量
>
> 关系模式 <-> 类型定义

第三章

三种完整性约束

1. **实体完整性**：确保每个表的每一行是唯一可标识的
2. **参照完整性**：要求参照关系的元组在特定属性上的取值等于被参照关系中某个元组在该属性上的取值
3. **用户定义完整性**：针对特定应用领域的数据约束条件

另一种说辞：

三个基本的完整性约束：

- **主码约束** primary key $(A_1, A_2, \dots)$：属性 $(A_1, A_2, \dots)$构成关系的主码
- **外码约束** foreign key$(B_1, B_2, \dots)$ references s：属性$(B_1, B_2, \dots)$ 上的取值必须对应于 *关系s中某元组**在主码属性上*** 的取值
- **非空约束** not null：该属性上不允许空值

约束

- unique约束：指出属性A1,A2,...Am形成了一个候选码，但是可以为null
- check子句：指定一个谓词P，关系中的每个元组都必须满足谓词P

SQL语言与关系代数对应

关系代数是**集合运算**！SQL不是

1. SELECT **对应** π（投影）

   但是并不等价，因为 select 会返回重复的行，但是**集合运算**不会返回重复的行。

2. FROM **对应 笛卡尔积（×）**
3. WHERE **对应 σ（选择）**
4. as **对应** ρ

SQL语言语法

- union：集合并
- intersect：集合交
- except：集合减

第四章

视图：可看作一个虚表，并不真实存在，计算时仍要从基本表中计算

物化视图：如果用于定义视图的实际关系改变，视图也跟着修改

- 保持物化视图一直在最新状态的过程称为物化视图维护 或 视图维护

视图更新：一个视图是可更新的，如果其满足以下条件

- from子句中只有一个数据库关系
- select子句只包含关系的属性名，不包含表达式、聚集或distinct声明
- 任何没有出现在select子句中的属性都没有not null约束，同时也不是主码的一部分
- 查询中不含有group by或having子句

事务：查询和更新语句的序列组成

事务结束标志：提交与回滚

第五章

触发器：一条语句，当对数据库做修改时，它自动被系统执行

- 设置要求：
  - 指明**什么条件**下执行触发器
  - 指明触发器**执行的动作**

第六章

关系代数是一种过程化查询语言

sql中：

- select语句可看作投影，使用 π
- where语句可看作选择，使用 σ
- from语句可看作选择，使用 $\times$

外连接：连接运算的扩展，它在结果中创建带空值的元组，以此来保留在连接中丢失的那些元组

第七章

E-R数据模型采用三个基本概念：实体集、联系集、属性

映射基数：表示一个实体通过一个联系集能关联的实体的个数

- 在联系集连线中，一的一端有箭头，多的一端无箭头（双线：每个实体都参与到至少一个联系中）

二元联系集主码选择：

- 多对多：实体集主码的并集
- 一对多or多对一：“多方”的主码是最小的超码，用作主码
- 一对一：任一参与联系集的实体集的主码构成最小超码，也可以当做主码

特化：在实体集内部进行分组

概化：一个高层实体集和低层实体集

第八章

无损分解：

如果用两个关系模式$r_1(R_1)$和$r_2(R_2)$替代$r(R)$时没有信息损失，则我们称该分解是**无损分解**

不是无损分解的分解称为**有损分解**

R_1和R_2是R的无损分解，如果以下函数依赖中至少有一个属于F^+：

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

example:

为举例说明，考虑模式

$$inst_dept\ (ID, name, salary, dept_name, building, budget)$$

我们在 8.1.2 节中将其分解为 *instructor* 和 *department* 模式：

$$instructor\ (ID, name, dept_name, salary)$$
$$department\ (dept_name, building, budget)$$

考虑这两个模式的交集，即 *dept_name*。我们发现，由于 $dept_name \rightarrow dept_name, building, budget$，因此满足无损分解条件。

函数依赖集的闭包：

- Armstrong公理：
  - **自反律**：若α为一属性集且$\beta \subseteq \alpha$，则$\alpha \rightarrow \beta$成立
  - **增补律**：若$\alpha \rightarrow \beta$成立且γ为一属性集，则$\gamma\alpha \rightarrow \gamma\beta$
  - **传递律**：若$\alpha \rightarrow \beta$和$\beta \rightarrow \gamma$成立，则$\alpha \rightarrow \gamma$成立
- 计算F+算法：用以上三个公理不断扩展函数依赖集，直到不再发生变化

属性集的闭包：

- 令α为一个属性集，F下被α确定的所有属性的集合称为F下α的闭包，记为α^+

- 例子：
  - $R = (A, B, C, G, H, I)$
  - $F = \{A \rightarrow B$
    $A \rightarrow C$
    $CG \rightarrow H$
    $CG \rightarrow I$
    $B \rightarrow H\}$
  - $(AG)^+$
    1. $result = AG$
    2. $result = ABCG \quad (A \rightarrow C \text{ and } A \rightarrow B)$
    3. $result = ABCGH \quad (CG \rightarrow H \text{ and } CG \subseteq AGBC)$
    4. $result = ABCGHI \quad (CG \rightarrow I \text{ and } CG \subseteq AGBCH)$

正则覆盖：

- 正则覆盖：不含无关属性的函数依赖集，并且左半部分都是唯一的
- 无关：如果去除函数依赖中的一个属性不改变该函数依赖集的闭包，则称该属性是无关的
- 如何判断一个属性是否无关：
  - 左边属性：将该属性去除，计算**旧F下**（未删除属性前的函数依赖集），去除该属性后的α是否仍能推出β，可以则说明该属性无关
  - 右边属性：将该属性去除，计算**新F下**（删除属性后的函数依赖集），α是否仍能推出原β，可以则说明该属性无关
  - example:

    例如，假定 F 包含 $AB \rightarrow CD$、$A \rightarrow E$ 和 $E \rightarrow C$。为检验 C 在 $AB \rightarrow CD$ 中是否是无关的，我们计算 $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$ 下 AB 的属性闭包。闭包为 $ABCDE$，包含 CD，所以我们推断出 C 是无关的。

保持依赖：

- 保持依赖分解：F在R_i上的限定是F^+中所有只包含R_i中属性的函数依赖的集合F_i。令$F' = F_1 \cup F_2 \cup \ldots \cup F_n$，若分解具有性质$F'^+ = F^+$，则该分解被称为保持依赖的分解。
- 算法：

  $result = \alpha$
  repeat
  　　for each 分解后的 R_i
  　　$t = (result \cap R_i)^+ \cap R_i$
  　　$result = result \cup t$
  until ($result$ 没有变化)

- 注意：对于每一个依赖，都要对每一个子模式计算，只要$result$包含了β的所有属性，则该函数依赖是保持依赖的。

BCNF分解算法：

- 算法
  - 判断每一个依赖是否平凡函数依赖（1. $\beta \subseteq \alpha$，2. α是模式R的超码），若不满足则将其进行分解，否则看下一个函数依赖。
  - 如何判断子模式是否满足BCNF？只要有一个子集违反，则说明不满足BCNF，将子模式进行分解。
    - **对于 R_i 中属性的每个子集 α，确保 α^+（F 下 α 的属性闭包）要么不包含 $R_i - \alpha$ 的任何属性，要么包含 R_i 的所有属性。**

    即，如果存在这样的函数依赖 $\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$ 在F^+中，说明R_i违反BCNF

3NF分解算法：

- 算法：求出函数依赖集的正则覆盖，对每一个依赖将其当做集合，最后的分解结果是模式的集合
- 注意：若以上子模式都没有候选码，则需要另外添加一个模式，其中包含R的任一候选码

多值依赖：

简单理解为一对多

第十章

通过冗余提高可靠性

镜像：复制每张磁盘

通过并行提高性能

通过在多张磁盘上进行**数据拆分**来提高传输速率

有 **比特级拆分** 和 **块级拆分** 等方式

两个主要目的：

1. 负载平衡多个**小的**访问操作（块访问），以提高这种访问操作的**吞吐量**
2. 并行执行**大的**访问操作，以减少**响应时间**

> 每个磁盘都能参与

变长记录

空位图：表示记录的哪个属性是空值

分槽的页结构：用以存储变长记录

块头信息：

- 块头中*记录项的数量*
- 块中 *自由空间的末尾* 处
- 一个由包含*每条记录的位置和大小* 的项组成的数组

文件中记录的组织

文件：记录的一个序列（逻辑上）

- **堆文件组织**：记录可以存放在文件的任意位置，不再移动
- **顺序文件组织**：按搜索码顺序对记录排序
- **多重聚簇文件组织**：多个关系记录存在同一个文件中（用以计算连接join）
- **B+树文件组织**：
- **散列文件组织**

缓冲区替换策略

- **LRU**：最近最少使用
- **MRU**：最近最常使用

第十一章

索引类型

- **顺序**索引
- **散列**索引

顺序索引类型

- **稠密**索引：**每个**搜索码值都有一个索引项
- **稀疏**索引：只为**某些**搜索码值建立索引项

静态散列

基于散列技术（**搜索码值 -> 桶**）的文件组织，使我们能够避免访问索引结构

动态散列

允许**散列函数动态改变**，以适应数据库增大或缩小的需要

动态散列与静态散列比较

可扩充散列

- 优点：

  性能不随文件的增长而降低

  空间开销是最小的，尽管桶地址表带来了额外的开销，但该表较小

- 缺点：

  查找涉及一个**附加的间接层**，对性能有微小的影响

第十二章

使用文件扫描和索引的选择 开销

	算　法	开　销	原　因
A1	线性搜索	$t_s + b_r * t_T$	一次初始搜索加上 b_r 个块传输，b_r 表示在文件中的块数量
A1	线性搜索，码属性等值比较	平均情形 $t_s + (b_r/2) * t_T$	因为最多一条记录满足条件，所以只要找到所需的记录，扫描就可以终止。在最坏的情形下，仍需要 b_r 个块传输
A2	B⁺树主索引，码属性等值比较	$(h_i + 1) * (t_T + t_s)$	（其中 h_i 表示索引的高度）。索引查找穿越树的高度，再加上一次 I/O 来取记录；每个这样的 I/O 操作需要一次搜索和一次块传输
A3	B⁺树主索引，非码属性等值比较	$h_i * (t_T + t_s) + b * t_T$	树的每层一次搜索，第一个块一次搜索。b 是包含具有指定搜索码记录的块数。假定这些块是顺序存储（因为是主索引）的叶子块并且不需要额外搜索
A4	B⁺树辅助索引，码属性等值比较	$(h_i + 1) * (t_T + t_s)$	这种情形和主索引相似
A4	B⁺树辅助索引，非码属性等值比较	$(h_i + n) * (t_T + t_s)$	（其中 n 是所取记录数。）索引查找的代价和 A3 相似，但是每条记录可能在不同的块上，这需要每条记录一次搜索。如果 n 值比较大，代价可能会非常高

外部排序归并的代价分析

- 磁盘块传输：

  磁盘块传输 总数： $2b_r + b_r(2 * \lceil log_{\lfloor M/b_b \rfloor - 1}(b_r/M) \rceil - 1)$

- 磁盘搜索：

  磁盘搜索 总次数： $2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil (2\lceil log_{\lfloor M/b_b \rfloor - 1}(b_r/M) \rceil - 1)$

  （本质上就是磁盘块传输的**“+”后面**的**M**和b_r **都除以**b_b ）

连接运算代价

- **嵌套循环连接**：

  > 最坏情况：缓冲区只能**容纳每个关系的一个数据块**
  >
  > 最好情况：缓冲区能同时容纳两个关系

	最坏情况	最好情况
块传输	$b_r + n_r * b_s$ （存r+存s）	$b_r + b_s$
磁盘搜索	$b_r + n_r$ （找r记录 + 每次遍历s需要找一次s的起始位置）	2

  示例：

  使用下面的表达式做示例：$student \bowtie takes$

  - student记录数：$n_{student} = 5000$
  - student磁盘块数：$b_{student} = 100$
  - takes记录数：$n_{takes} = 10000$
  - takes磁盘块数：$b_{takes} = 400$

  1. student外层，takes内层：

     块传输次数：100+ 5000 * 400 = 2000100

     磁盘搜索次数：100 + 5000 = 5100

  2. takes外层，student内层：

块传输次数：400 + 10000 * 100 = 1000400

磁盘搜索次数：400 + 10000 = 10400

- **块嵌套循环连接**：

	最坏情况	最好情况
块传输	$b_r + b_r * b_s$	$b_r + b_s$
磁盘传输	$2b_r$	2

示例：

使用下面的表达式做示例：$student \bowtie takes$

- student记录数：$n_{student} = 5000$
- student磁盘块数：$b_{student} = 100$
- takes记录数：$n_{takes} = 10000$
- takes磁盘块数：$b_{takes} = 400$

1. student外层，takes内层：

   块传输次数：100 + 100 * 400 = 40100

   磁盘搜索次数：100 + 100= 200

2. takes外层，student内层：

   块传输次数：400 + 400 * 100 = 40400

   磁盘搜索次数：400 + 400 = 800

第十三章

查询优化

如果两个关系代数表达式在每个合法的数据库实例上都会**产生相同的元组集**，则这两个表达式是**等价**的。

目标：尽量执行选择和投影操作

左深连接顺序用于流水线计算特别方便

重要的等价规则：

1.

连接运算满足交换律

连接运算（Join Operation）满足**交换律**，即两个关系的连接顺序可以互换，结果不变。

$$R \bowtie S = S \bowtie R$$

2.

选择条件只涉及左侧关系的属性

$$\sigma_\theta(R \bowtie S) = \sigma_\theta(R) \bowtie S$$

前提：选择条件 θ 只涉及关系 R 的属性。

第十四章

事务：构成单一逻辑操作单元的操作集合
数据库系统要么执行整个事务，要么属于该事务的操作一个也不执行

事物的并发调度

并发的好处：
提高吞吐量和资源利用率
减少等待时间

当多个事务并发执行时，隔离性可能被违背，这导致即使每个单独的事务都是正确的，但数据库的一致性还是会被破坏。

调度：描述的执行顺序，定义了指令在系统中的执行的时间顺序。必须包含所有事务的全部指令，并且这些指令必须保持它们原本事务的顺序。

结论：并非所有的并发执行都能得到正确的状态（即保证了数据库的一致性）。

并发的问题

并发的一些问题

1. 脏读： 一个事务读取了另一个事务尚未提交的数据。如果第二个事务回滚，第一事务读取到的数据就会无效。这会导致数据不一致。
2. 不可重复读： 在同一事务中，两次读取同一数据得到的结果不同，原因是另一事务在期间修改了数据。
3. 幻读：是指在一个事务内，多次查询同一范围的数据时，得到的结果集不一致。这种现象通常发生在并发事务中，当一个事务在读取数据时，另一个事务插入了新的数据行，导致第一个事务在后续读取时发现了新的行。

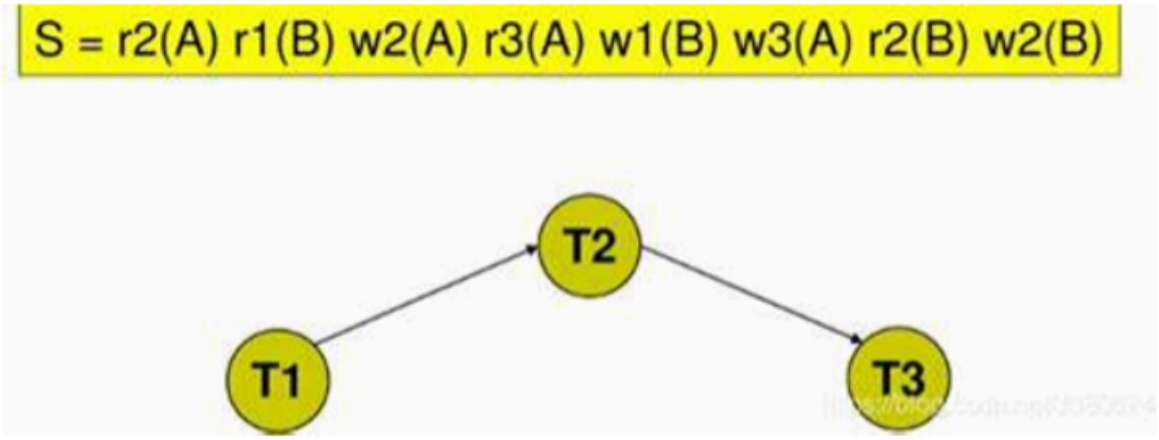
可串行化调度

冲突可串行化指的是一个并发事务调度可以通过交换不冲突的操作顺序，转化为某个串行调度，且两个调度的执行结果完全相同。

优先图（有向图）

结论：如果优先图中无环，则该调度是可冲突串行化的。

对于同一数据，只要T2对于该数据的操作在于T1对于该数据的操作之前，（除了两个都为读操作），则插入由T2为起点，T1为终点的有向边进入优先图。代表T2执行完后，T1才能执行。



要记得起点是先执行的！！

事务的可恢复性调度

可恢复调度是指一种事务调度方式，其中如果一个事务T1依赖于另一个事务T2的修改（即T1读取了T2修改的数据），那么 T2必须在T1提交之前先提交。

这确保了如果T2发生故障并回滚，依赖于它的事务T1也可以安全地回滚，避免数据不一致。

无级联调度是一种更严格的可恢复的调度，它在前者的基础上，一定要确保T2已经提交后，T1才能读取。这样，即使T2失败，T1也不会收到影响，因为T1值只依赖于已提交的数据

隔离性级别

- 可串行化
- 可重复读
- 已提交读
- 未提交读

第十五章

封锁协议

- 共享锁：读取数据
- 排他锁：写数据用

只有共享锁和共享锁之间是相容的，即共享锁可以被多个事务持有，但排他锁必须一直等到这些事务都释放共享锁才能获得

避免饿死的加锁方式

饿死：事务总是都得不到锁，无法进展

1. 不存在数据项Q上持有与M型锁冲突的锁的其它事务
2. 不存在等待对数据项Q加锁且先于事务Ti申请加锁的事务

此时一个加锁请求就不会被其后的加锁申请阻塞

死锁：事务 在等待事务释放关于数据 X 的锁，与此同时，事务在等待事务释放关于数据 Y 的锁，两个事务之间互相等待，就发生了死锁

两阶段协议

- 增长阶段：事务可以获得锁，但不能释放锁
- 缩减阶段：事务可以释放锁，但不能获得锁

锁的升级只能发生在增长阶段，锁的降级只能发生在缩减阶段

严格两阶段封锁可以避免级联回滚，要求所有排他锁必须在事务提交后才能释放强

两阶段封锁要求事务提交之前不得释放任何锁

树形协议：

可用的加锁指令只有lock-x

规则如下:

事务T在首次加锁时可以对任何数据项进行

然后，他对于数据项q可以加锁的前提是他持有q父节点的锁，解锁没有要求

数据项被T解锁后，T不能再次对该数据项加锁。

特性：

- 所有满足树形协议的调度是冲突可串行化的。
- 不产生死锁，所以不会回滚。

死锁处理

两种方法：

- 死锁预防：永不进入死锁
    - 对加锁请求进行排序或要求同时获得所有的锁来保证不会发生循环等待
    - 每当等待有可能导致死锁时，进行回滚而不是等待加锁
    - 申请锁的事务至多等待一段给定的时间，若超时则回滚并重启
- 死锁检测和死锁恢复机制

  等待图有环则会产生死锁

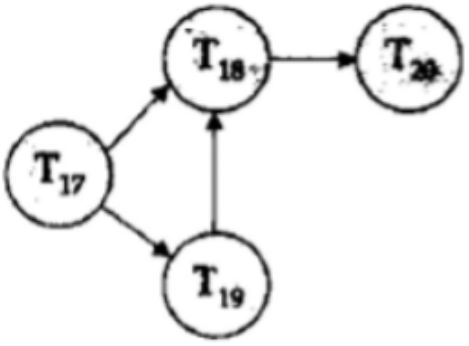


图 15-13　无环等待图

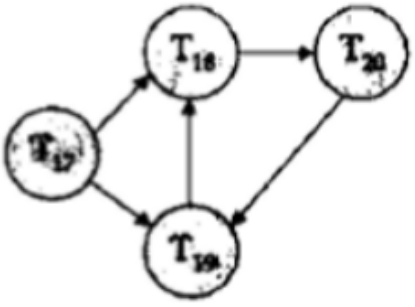


图 15-14　有一个环的等待图

死锁恢复

选择牺牲者：决定回滚哪一个

回滚：必须要决定事务回滚多远

防止饿死：保证一个事务被选为牺牲者的次数有限

第十六章

恢复系统

日志的恢复：

正像扫描日志文件，找出在故障发生前已经提交的事务，将其事务标识记入REDO队列，同时找出故障发生时尚未完成的事务，将其事务标识记入UNDO队列；对UNDO队列中的各个事务进行撤销处理；对REDO队列中的各个事务进行重做处理。

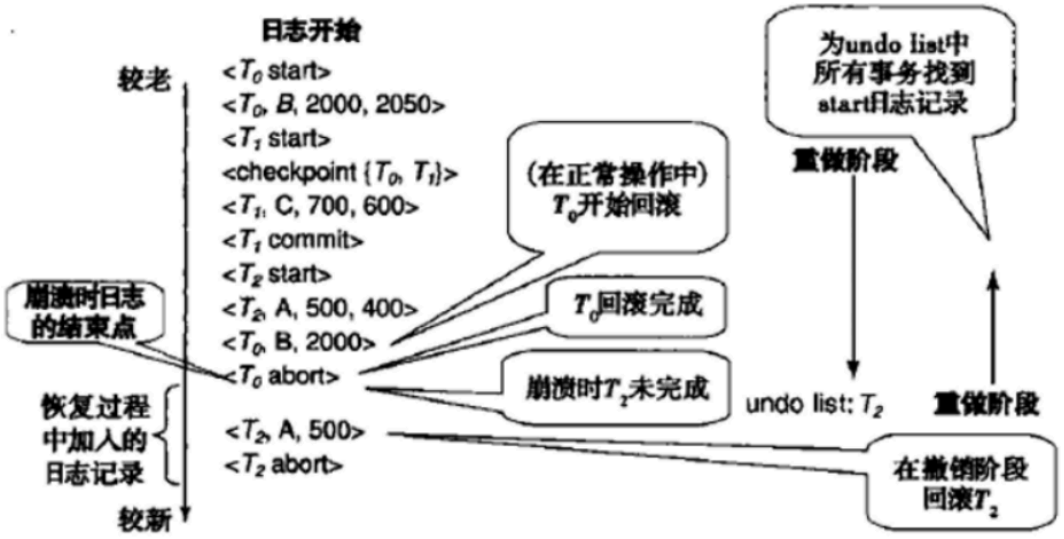


图 16-5　记录在日志中的动作和恢复中的动作的例子

当从崩溃中恢复时，在重做阶段，系统对最后一个检查点记录之后的所有操作执行 redo。在这个阶段中，undo-list 初始时包含 T_0 和 T_1；当 T_1 的 **commit** 日志记录被发现时，T_1 先被从表中去掉，而当 T_2 的 **start** 日志记录被发现时，T_2 被加到表中。当事务 T_0 的 **abort** 日志记录被发现时，T_0 被从 undo-list 中去掉，只剩 T_2 在 undo-list 中。撤销阶段从尾端开始反向扫描日志，当发现 T_2 更新 A 的日志记录时，将 A 恢复成旧值，并往日志中写一个 redo-only 日志记录。当发现 T_2 开始的日志记录时，就为 T_2 添加一条 **abort** 记录。由于 undo-list 不再包含任何事务了，因此撤销阶段终止，恢复完成。